

Entregable 1

Taller de Aprendizaje Automático

Estéfano Bargas

Abril 2024

1. Demanda de bicicletas

1.1.

El hiperparametro seleccionado fue *max_depth* que regula la profundidad máxima entre una hoja y la raíz del árbol. Se realizo una búsqueda grid con validación cruzada en un rango de 1 a 15, decidido arbitrariamente. En la [Figura 1](#) se gráfica el RMSLE de validación en función del hiperparametro. El valor óptimo encontrado fue de 9.

1.2.

En general los métodos de ensamble están hechos para mejorar el rendimiento de un modelo débil a partir de agregar los resultados de modelos diversos. En particular para random forests la idea es minimizar la varianza (a cambio de un leve empeoramiento del sesgo) al entrenar arboles en diferentes subconjuntos de los datos de entrenamiento (técnica llamada *bagging*), lo cual resulta en arboles débiles y diversos cuyos resultados se promedian (o en caso de clasificación, se utiliza votación). Existen otros metodos para agregar los resultados.

Los resultados de mi notebook arrojan que mientras un solo árbol (luego de ajuste fino) obtiene un RMSLE en validación cruzada de promedio 0,49 y desviación estándar 0,013, un random forest obtiene un promedio 0,38 y desviación 0,007, indicando una mejora de rendimiento y menor varianza en distintos subconjuntos de validación.

1.3.

El mejor estimador del taller 3 fue un random forest con gradient boosting y tuneado con búsqueda bayesiana, que resulto en un RMSLE de 0.40374 en test y 0.29 en validación. El del taller 5 fue una red neuronal tuneada con Hyperband, con una forma final de 8 capas ocultas con 32 neuronas cada una, que resulto en un RMSLE de 0.55972 a pesar de tener un error de 0.08 en validación, lo cual puede indicar sobreajuste. Debido a esto se intento regularizarla mediante dropout entre cada capa a un rate de 0.05 y si bien el error en validación (0.14) se acerco un poco mas al de test, este ultimo empeoro. Se jugo también con las proporciones de los datos de entrenamiento y se entreno con 200 epochs y early-stopping, sin éxito. Seria necesaria mas ingeniería de características. Los pipelines de ambos tienen en común las columnas nuevas creadas (para la hora, día de la semana y mes) y las diferencias son que el random forest entrena con los logaritmos de los valores objetivos (la red no), y que la red normaliza los datos (el forest no).

1.4.

En la [Figura 2](#) se encuentra una gráfica de fuerza de cada feature para el dato solicitado. La figura fue generada con la librería *SHAP*. En general las features mas relevantes son las características instantáneas del día: la hora, la temperatura y la sensación térmica, revelando que las condiciones que hacen un día agradable tienen gran efecto en la cantidad de bicicletas rentadas. Se puede ver que el hecho de ser un día sábado afecta negativamente a las rentas, lo que sugiere que la mayoría de ciclistas sale entre semana.

2. Detección de anomalías

2.1.

Bajo las definiciones del libro del curso, denominar al problema como "deteccion de anomalias" no es preciso ya que trabajamos con modelos entrenados en datos limpios (sin outliers) y luego utilizamos estos para detectar *novedades*. Lo mas correcto seria llamarle un problema de "deteccion de novedades", pues en un problema de anomalias no asumimos tener un conjunto de datos limpio. Es importante tener presente esta distincion ya que nos puede llevar a elecciones diferentes para la solucion.

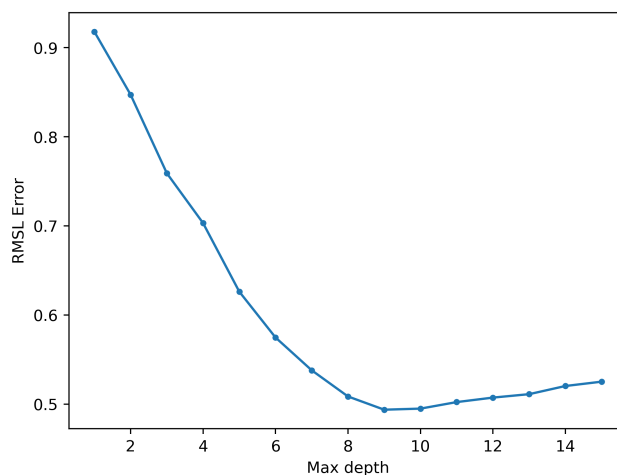


Figura 1: RMSLE de validación en función del hiperparametro max_depth.

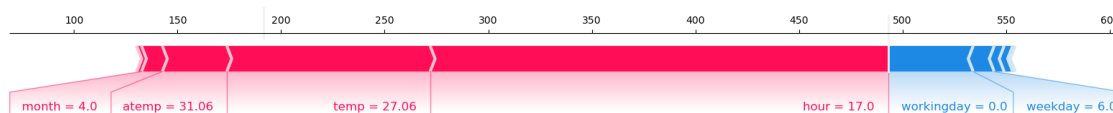


Figura 2: Gráfico de fuerza del mejor estimador para la demanda de bicicletas, generado con SHAP.

Una vez aclarada esta diferencia, es correcto decir que el problema es de detección de novedades ya que contamos con un conjunto de entrenamiento limpio (los servicios funcionando normalmente) y conjuntos de test con novedades (servicios siendo atacados).

2.2.

En el [Fragmento 1](#) se encuentra el código correspondiente al Pipeline de preprocesamiento utilizado, y la definición del imputer. Se utilizan ambos componentes para armar el pipeline final con el modelo de aprendizaje. El imputer rellena los

```
# rellena con 0 y 'missing_value'
imputer = SimpleImputer(strategy='constant', fill_value=None)
imputer.set_output(transform="pandas")

preprocess = ColumnTransformer([
    ('encode', OneHotEncoder(
        drop='if_binary',
        sparse_output=False,
        handle_unknown='ignore'
    ), discrete_cols),
    ('std', StandardScaler(), continuous_cols)
], remainder='drop')
preprocess.set_output(transform="pandas")
```

Fragmento 1: Pipeline de preprocesamiento para detección de anomalías.

valores faltantes con 0 en caso de ser numéricos y con "missing_value" en caso de ser textuales, se eligió esta opción ya que si rellenamos con valores estándar, estaríamos dándole características normales a un servicio cuando no estamos seguros de que ese sea el caso, lo que llevaría a un falso negativo. Dentro del pipeline de preprocesamiento se encuentra un one-hot encoder que ignora categorías que no estaban presentes en el conjunto de entrenamiento, y produce una sola columna en caso de tener una categoría binaria. Luego se estandarizan los datos para que al realizar un análisis de componentes principales ninguna columna presente mas varianza que otra debido a su escala en vez de su distribución. Al final se ignoran las columnas que no estaban presentes en el entrenamiento.

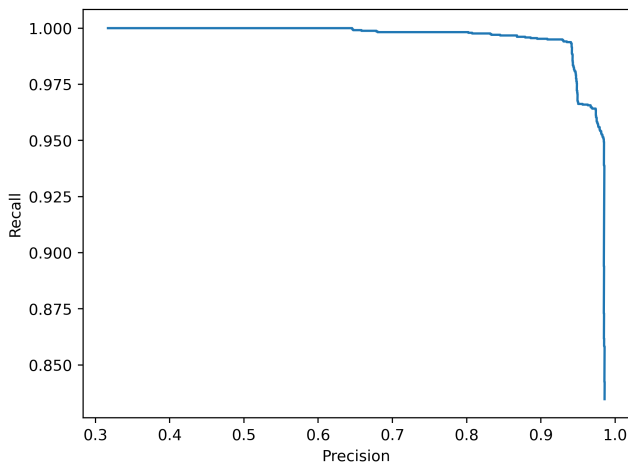


Figura 3: Curva Precision-Recall para la estrategia PCA en el conjunto de validación.

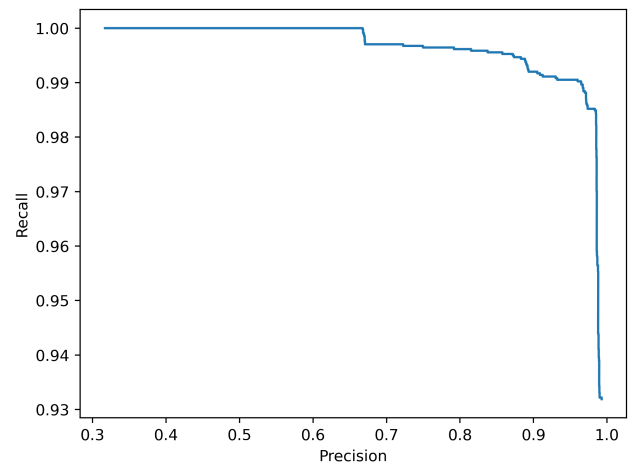


Figura 4: Curva Precision-Recall para la estrategia KMeans en el conjunto de validación.

2.3.

Las dos estrategias a comparar son reconstrucción con PCA y KMeans. La primer estrategia consta en obtener las componentes principales que preservan el 99 % de la variación de los datos normales (que resultan en 30) y utilizando este subespacio vectorial, proyectar y reconstruir datos anómalos y clasificar según el RMSE contra los datos originales (los datos anómalos presentan un error mucho mayor ya que no pertenecen al subespacio de los datos normales). La segunda consta en, primero obtener el numero optimo de clusters (que resulta en 2), el cual maximiza el *sillhouette score*, y luego agrupar los datos normales. Para clasificar se mide el promedio de las distancias entre los centroides y el dato a clasificar, un dato anómalo presenta mucha mayor distancia.

En la [Figura 3](#) se encuentra la curva precision-recall para PCA y en la [Figura 4](#) para KMeans obtenida de clasificar el conjunto de validación. Dado que el enfoque del problema es minimizar la cantidad de falsas alarmas, nos enfocamos en maximizar la precisión. Bajo esto concluimos que PCA es la mejor estrategia ya que la precisión decae mas lentamente en función al recall. Se realizaron los mismos experimentos utilizando GMM bayesiano y midiendo la verosimilitud pero los resultados fueron insuficientes (precisión máxima menor al 40 %), probablemente debido a que los datos no siguen una distribución uniforme.

2.4.

Para obtener el punto de operación de PCA graficamos sus curvas precision y recall en función de los thresholds en la [Figura 5](#) para los datos de validación. Priorizando la precision, elegi el punto de operacion para un RMSE de 0.18, lo cual resulta en una precision de 98.5 % y un recall de 90.8 % en validación. Este es el punto aproximado en el cual la pendiente de la precisión se vuelve menor a la del recall en valor absoluto.

2.5.

Para el conjunto de test se obtuvo una precisión del 99.3 % pero un recall de 66.1 %, indicando que el punto de operación elegido en validación no es adecuado para que el modelo generalice. En la [Figura 6](#) se observa que un mejor threshold esta en el entorno de 0.12 (favorece la precisión), posterior a ese valor el recall baja considerablemente. El problema de fondo puede ser que el subespacio de datos normales encontrado para el conjunto de entrenamiento sea diferente al de los datos de test, o que el conjunto de test contenga ataques que en promedio producen menos síntomas en los servicios, necesitando entonces disminuir el threshold.

Si ajustamos el threshold en 0.12, obtenemos una precisión de 99.0 %, un recall de 93.4 % y los aciertos por categoría mostrados en la [Figura 7](#). El clasificador tiene problemas para detectar ataques R2L, en particular podemos observar una caída brusca en el recall para un threshold en el entorno de 0.05, y efectivamente si lo ajustamos a 0.04 obtenemos los aciertos de la [Figura 8](#). Para este valor la precisión cae a 92.6 % (observe los aciertos de los datos normales) pero mejoramos mucho la detección de los ataques R2L. Esto sugiere que los ataques R2L tienen síntomas diferentes al resto de ataques y que lo optimo seria aplicar un threshold particular para ellos, incluso uno para cada categoría.

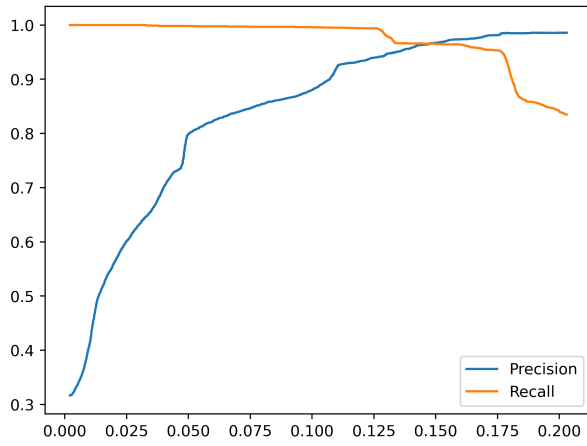


Figura 5: Curvas de precision y recall en función del threshold para la estrategia por reconstrucción con PCA en el conjunto de validación.

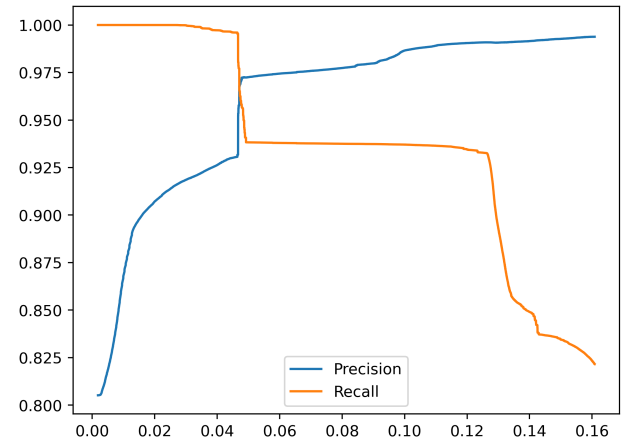


Figura 6: Curvas de precision y recall en función del threshold para la estrategia por reconstrucción con PCA en el conjunto de test.

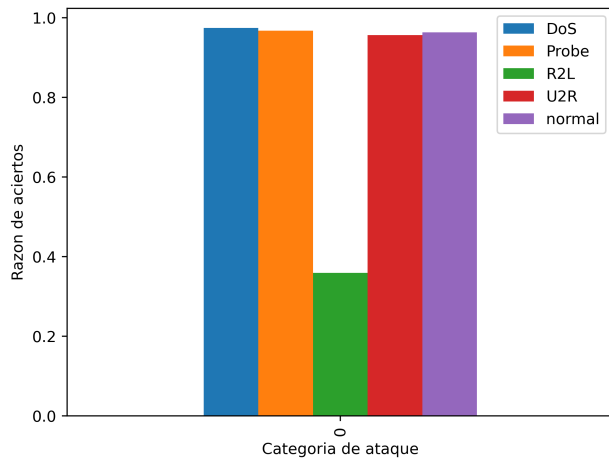


Figura 7: Razon de aciertos para cada categoría de ataque del conjunto de test para un threshold de 0.12.

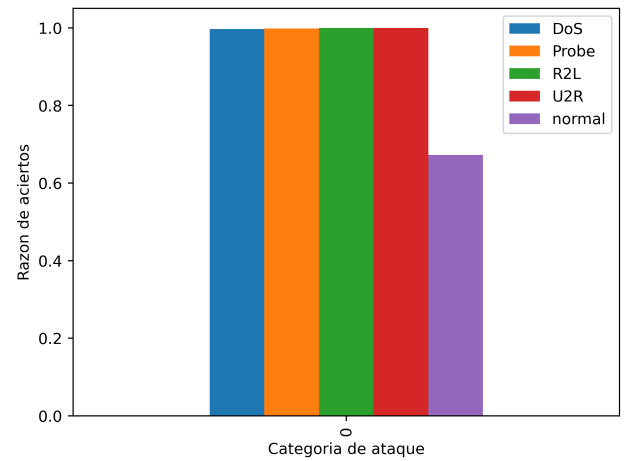


Figura 8: Razon de aciertos para cada categoría de ataque del conjunto de test para un threshold de 0.04.